

AI INTELLIGENCE PIPELINE

Technical Architecture - AI Engineer Demo

1. SYSTEM OVERVIEW

The AI Intelligence Pipeline is a modular, fault-tolerant ingestion system for research papers, startups, products, jobs and AI news.

External Sources -> Async Crawlers -> Validation -> Entity Resolution -> Freshness Filter -> LLM Extraction -> Enrichment -> Storage -> API

2. SCALE STRATEGY - 500,000+ RECORDS

The crawler layer uses asynchronous I/O and independent source adapters. Scaling is achieved through horizontal workers rather than application rewrites.

- Partition sources into independent crawl jobs.
- Distribute jobs through Redis/Kafka or an equivalent queue.
- Run multiple crawler workers concurrently.
- Apply per-domain concurrency and rate limits.
- Persist records incrementally in batches.
- Use checkpoints for restart and resume.
- Cache processed URLs and identifiers.
- Use database indexes and uniqueness constraints.
- Scale workers horizontally using containers/Kubernetes.

The architecture therefore scales by adding infrastructure capacity while source adapters remain unchanged.

3. RESEARCH PAPER VERTICAL

Research papers are collected from sources such as arXiv.

Required fields include title, authors, paper URL, publication timestamp, GitHub repository URL when explicitly associated, and current GitHub star count.

GitHub metrics are obtained from the GitHub API. LLMs are not used to invent source data. Every record retains its original source URL. Missing source information remains missing rather than fabricated.

4. FRESHNESS TRACKING

News and jobs must be published within the previous 24 hours.

Source Feed -> Date Extraction -> UTC Normalization
-> 24-Hour Freshness Check -> Fingerprint -> Deduplication -> Storage

Dates are extracted from RSS/Atom metadata and source-specific parsers.
Relative dates such as "2 hours ago" are normalized to absolute UTC.

For distributed crawlers, a shared fingerprint and database unique constraint prevents multiple crawler nodes from processing the same item.

5. LLM EXTRACTION

Fallback chain:

Gemini Flash -> Groq/Llama -> DeepSeek

413 HANDLING

- Estimate payload size before every request.
- Remove navigation and boilerplate.
- Preserve titles, dates, metadata and dense content.
- Chunk oversized documents.
- Process chunks independently.
- Merge validated structured results.
- Enforce a maximum payload threshold.

429 HANDLING

- Exponential backoff.
- Random jitter.
- Respect Retry-After when provided.
- Per-provider concurrency limits.
- Bounded retries.
- Provider failover.

Every LLM response is schema-validated. The source remains the authoritative data origin.

6. ENTITY RESOLUTION

Messy organization and product names are mapped to canonical entities.

'Open AI' / 'OpenAI Inc.' / 'OpenAI, Inc.' -> 'OpenAI'

Normalization includes case, whitespace, punctuation and corporate suffix normalization.

A canonical seed table is checked first. Fuzzy matching is used only with conservative thresholds.

Every mapping is auditable using raw name, canonical name, source URL, matching method and confidence.

7. ANTI-BOT AND JAVASCRIPT-HEAVY SOURCES

The system does not attempt to bypass CAPTCHAs or defeat security controls.

For permitted JavaScript-rendered sources:

- Use Playwright Async when appropriate.
- Prefer official APIs and feeds.
- Respect robots.txt and site terms.
- Use conservative per-domain request rates.
- Cache successful responses.
- Use licensed or approved data access.

Cloudflare/Datadome protected sources should be accessed through permitted APIs, licensed sources, or approved browser access rather than CAPTCHA bypassing.

8. STORAGE STRATEGY

PRIMARY DATABASE - PostgreSQL

PostgreSQL stores canonical entities, source metadata, timestamps, relationships, validation state and audit logs.

It provides transactions, indexes, uniqueness constraints, strong consistency and scalable relational queries.

VECTOR STORAGE

pvector or a dedicated vector database can store embeddings for semantic research-paper search and recommendations.

GRAPH STORAGE

Neo4j or another graph database can model:

Startup -> Product
Paper -> Author
Paper -> GitHub Repository
Paper -> Topic
Company -> Job

PostgreSQL remains the system of record while vector and graph stores support specialized retrieval.

9. RELIABILITY / PRODUCTION READINESS

The pipeline isolates source failures so one unavailable source does not stop the complete ingestion process.

Key mechanisms: asynchronous I/O, bounded concurrency, retries with exponential backoff, checkpointing, incremental persistence, source failure isolation, schema validation, deterministic deduplication, structured logging, provider fallback and source traceability.

10. CURRENT DEMO IMPLEMENTATION

The repository demonstrates the research-paper vertical with a 1,000-paper processed dataset.

Implemented: arXiv collection, GitHub enrichment, keyword extraction, topic classification, research intelligence, scoring, trend analysis, recommendations, search indexing, similarity search and FastAPI.

Automated test suite: 76 tests passed.

The modular architecture allows additional startup, product, job and news adapters to be added independently.